Basic Debugging in C

Reading errors, finding logic bugs, print debugging and a first look at gdb

CPE101 Teaching Assistant

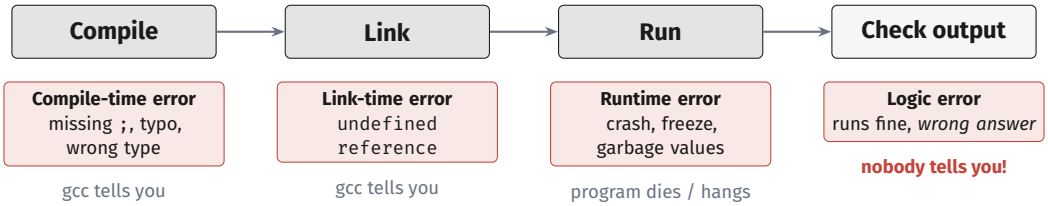
CPE101 Computer Programming
King Mongkut's University of Technology Thonburi (KMUTT)

Tutorial session

Today

- Four kinds of bugs
- Reading compiler errors
- Runtime and logic bugs
- Print debugging
- A first look at gdb

Where can things go wrong?



Left to right, bugs get **harder to find**. That's why we turn on warnings: they move bugs to the left, where gcc points straight at them.

TIP The scariest bug is a logic error: the program runs normally but gives the wrong answer, and nothing warns you.

The debugging mindset

A bug is a gap between

what you **think** the program does, and
what it **actually** does.

Debugging = **find where they first differ**.

1. **Reproduce**: find an input that shows the bug.
2. **Predict**: what *should* each variable be?
3. **Observe**: print / step through to see what it *is*.
4. **Narrow down**: the first line where they disagree.
5. **Fix** one thing, then test again.

Don't

- change random things until it works
- ignore warnings

Do

- test with **small** inputs you can check by hand
- read the **first** error message slowly
- explain the code line by line, out loud ("rubber duck")

Anatomy of a gcc message

```
ce2_undeclared.c:8:28: error: 'totl' undeclared (first use in this function); did you mean 'total'?
    8 |     printf("total = %d\n", totl);
      |                            ^~~~
      |                            total
```

file `ce2_undeclared.c`

line 8 go there first

column 28 where on the line

error or **warning** error = no program

the message often with a hint

Rule 1: fix the FIRST error first

One mistake (like a missing `;` or `}`) can cause 10 more messages after it. Fix the first, recompile, repeat.

Rule 2: the bug is at or BEFORE that line

gcc reports where it got *confused*. The real mistake is often on the line above.

TIP Read the first error fully before anything else. Fix one thing at a time, then recompile. Don't panic at long error messages.

Spot the bug #1

```c
#include <stdio.h>

int main(void)
{
    int x = 5
    printf("x = %d\n", x);
    return 0;
}
```

TIP The error says line 6, but the real mistake is on line 5 (missing `;`). Always check the line before as well.

Spot the bug #1

```c
#include <stdio.h>

int main(void)
{
    int x = 5
    printf("x = %d\n", x);
    return 0;
}
```

```
ce1_semicolon.c:6:5: error: expected ',' or ';' before 'printf'
    6 |     printf("x = %d\n", x);
      |     ^~~~~
```

TIP The error says line 6, but the real mistake is on line 5 (missing `;`). Always check the line before as well.

Spot the bug #1

```c
#include <stdio.h>

int main(void)
{
    int x = 5
    printf("x = %d\n", x);
    return 0;
}
```

```
ce1_semicolon.c:6:5: error: expected ',' or ';' before 'printf'
    6 |     printf("x = %d\n", x);
      |     ^~~~~
```

Missing `;` at the end of **line 5**. gcc only noticed when it reached `printf` on line 6.

Fix: `int x = 5;`

TIP The error says line 6, but the real mistake is on line 5 (missing `;`). Always check the line before as well.

Spot the bug #2

```c
#include <stdio.h>

int main(void)
{
    int total = 0;
    for (int i = 1; i <= 3; i++)
        total = total + i;
    printf("total = %d\n",
   totl);
    return 0;
}
```

Spot the bug #2

```c
#include <stdio.h>

int main(void)
{
    int total = 0;
    for (int i = 1; i <= 3; i++)
        total = total + i;
    printf("total = %d\n",
   totl);
    return 0;
}
```

```
ce2_undeclared.c:8:28: error: 'totl' undeclared (first use
  in this function); did you mean 'total'?
    8 |     printf("total = %d\n", totl);
      |                            ^~~~
      |                            total
```

Spot the bug #2

```c
#include <stdio.h>

int main(void)
{
    int total = 0;
    for (int i = 1; i <= 3; i++)
        total = total + i;
    printf("total = %d\n",
   totl);
    return 0;
}
```

```
ce2_undeclared.c:8:28: error: 'totl' undeclared (first use
  in this function); did you mean 'total'?
    8 |     printf("total = %d\n", totl);
      |                            ^~~~
      |                            total
```

Typo in a variable name. “Undeclared” means: *I’ve never seen a variable with this name.* Also check upper/lower case: `Total` $\neq$ `total`.

Fix: `totl` $\rightarrow$ `total`. gcc even suggested it.

Spot the bug #3

```c
#include <stdio.h>

int main(void)
{
    double x = 2.0;
    printf("sqrt = %f\n",
   sqrt(x));
    return 0;
}
```

Older gcc only gave a *warning* here. gcc 14+ makes it an error.

Spot the bug #3

```c
#include <stdio.h>

int main(void)
{
    double x = 2.0;
    printf("sqrt = %f\n",
   sqrt(x));
    return 0;
}
```

```
ce3_include.c:6:27: error: implicit declaration of
  function 'sqrt' [-Wimplicit-function-declaration]
    6 |     printf("sqrt = %f\n", sqrt(x));
      |                           ^~~~
ce3_include.c:2:1: note: include '<math.h>' or provide
  a declaration of 'sqrt'
  +++ |+#include <math.h>
```

Older gcc only gave a *warning* here. gcc 14+ makes it an error.

Spot the bug #3

```c
#include <stdio.h>

int main(void)
{
    double x = 2.0;
    printf("sqrt = %f\n",
   sqrt(x));
    return 0;
}
```

```
ce3_include.c:6:27: error: implicit declaration of
  function 'sqrt' [-Wimplicit-function-declaration]
    6 |     printf("sqrt = %f\n", sqrt(x));
      |                           ^~~~
ce3_include.c:2:1: note: include '<math.h>' or provide
  a declaration of 'sqrt'
  +++ |+#include <math.h>
```

"Implicit declaration" = you used a function that gcc has never heard of. Usually a **missing `#include`**, or a typo in the function name.

Fix: add `#include <math.h>` (and `-lm` on Linux).

Older gcc only gave a *warning* here. gcc 14+ makes it an error.

Spot the bug #4

```c
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 3; i++)
   {
        printf("%d\n", i);

    return 0;
}
```

Spot the bug #4

```c
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 3; i++)
    {
        printf("%d\n", i);

    return 0;
}
```

```
ce4_brace.c:9:1: error: expected declaration or statement
  at end of input
    9 | }
      | ^
```

Spot the bug #4

```c
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 3; i++)
    {
        printf("%d\n", i);

    return 0;
}
```

```
ce4_brace.c:9:1: error: expected declaration or statement
  at end of input
    9 | }
      | ^
```

"**At end of input**" = the file ended while a { was still open. The `for` loop's } is missing. gcc can only complain at the very end.

Tip: keep indentation tidy (most editors can auto-format). A missing brace then becomes obvious.

Spot the bug #5: it compiles... with warnings

```c
#include <stdio.h>

int main(void)
{
    float price;
    scanf("%d", &price);
    printf("price = %d\n",
   price);
    return 0;
}
```

```
$ ./ce5_format
21
price = 0
```

Spot the bug #5: it compiles... with warnings

```c
#include <stdio.h>

int main(void)
{
    float price;
    scanf("%d", &price);
    printf("price = %d\n",
   price);
    return 0;
}
```

```
$ ./ce5_format
21
price = 0
```

```
ce5_format.c:6:13: warning: format '%d' expects argument
  of type 'int *', but argument 2 has type 'float *'
    6 |     scanf("%d", &price);
ce5_format.c:7:22: warning: format '%d' expects argument
  of type 'int', but argument 2 has type 'double'
    7 |     printf("price = %d\n", price);
      |                      int   double
      |                     %f
```

Spot the bug #5: it compiles... with warnings

```c
#include <stdio.h>

int main(void)
{
    float price;
    scanf("%d", &price);
    printf("price = %d\n",
   price);
    return 0;
}
```

```
$ ./ce5_format
21
price = 0
```

```
ce5_format.c:6:13: warning: format '%d' expects argument
  of type 'int *', but argument 2 has type 'float *'
    6 |     scanf("%d", &price);
ce5_format.c:7:22: warning: format '%d' expects argument
  of type 'int', but argument 2 has type 'double'
    7 |     printf("price = %d\n", price);
      |                      int   double
      |                     %f
```

The **format specifier must match the type**:

type	scanf	printf
int	%d	%d
float	%f	%f
double	%lf	%f
char	%c (" %c")	%c

Bug: = instead of ==

```c
int score = 40;
if (score = 100)
    printf("Perfect score!\n");
else
    printf("Keep trying\n");
```

Output:

TIP = assigns, == compares. Inside an `if` condition you need ==.

Bug: = instead of ==

```c
int score = 40;
if (score = 100)
    printf("Perfect score!\n");
else
    printf("Keep trying\n");
```

Output:

```
Perfect score!
```

TIP = assigns, == compares. Inside an `if` condition you need ==.

Bug: = instead of ==

```c
int score = 40;
if (score = 100)
    printf("Perfect score!\n");
else
    printf("Keep trying\n");
```

Output:

```
Perfect score!
```

`score = 100` **assigns** 100 to `score`. The value of the whole expression is `100`, which is non-zero, so it counts as **true**.

`=` assignment: “make it equal”
`==` comparison: “is it equal?”

Fix: `if (score == 100)`

`-Wall` warns: *suggest parentheses around assignment used as truth value.*

TIP = assigns, == compares. Inside an `if` condition you need ==.

Bug: uninitialized variable (garbage value)

```c
int sum;
for (int i = 1; i <= 5; i++)
    sum += i;
printf("sum = %d\n", sum);
```

Expected `sum = 15`. **Got:**

TIP An uninitialised local variable holds a “garbage value”. Accumulators (sum, count) must start at 0; running products start at 1.

Bug: uninitialized variable (garbage value)

```c
int sum;
for (int i = 1; i <= 5; i++)
    sum += i;
printf("sum = %d\n", sum);
```

Expected `sum = 15`. **Got:**

```
sum = 32774
```

(a different number on another machine or another run)

TIP An uninitialised local variable holds a “garbage value”. Accumulators (sum, count) must start at 0; running products start at 1.

Bug: uninitialized variable (garbage value)

```c
int sum;
for (int i = 1; i <= 5; i++)
    sum += i;
printf("sum = %d\n", sum);
```

Expected `sum = 15`. **Got:**

```
sum = 32774
```

(a different number on another machine or another run)

A local variable starts with whatever bits were left in that memory: a **garbage value**. `sum += i` adds onto garbage.

Fix: `int sum = 0;`

Sneaky: plain `-Wall` at `-O0` did **not** warn here. `gcc -Wall -O2` does:

```
warning: 'sum' is used uninitialized [-Wuninitialized]
```

TIP An uninitialised local variable holds a “garbage value”. Accumulators (sum, count) must start at 0; running products start at 1.

Bug: integer division

```c
int total = 7, n = 2;
float avg = total / n;
printf("avg = %.2f\n", avg);
```

Expected `3.50`. **Got:**

TIP `7/2` gives `3`, not `3.5`, because it is integer division (relates to Q25 in the mock: use `1.0` instead of `1`).

Bug: integer division

```c
int total = 7, n = 2;
float avg = total / n;
printf("avg = %.2f\n", avg);
```

Expected `3.50`. **Got:**

```
avg = 3.00
```

TIP `7/2` gives `3`, not `3.5`, because it is integer division (relates to Q25 in the mock: use `1.0` instead of `1`).

Bug: integer division

```c
int total = 7, n = 2;
float avg = total / n;
printf("avg = %.2f\n", avg);
```

Expected `3.50`. **Got:**

```
avg = 3.00
```

`int / int` gives an **int**: the fraction is thrown away *before* the result is stored in the `float`.

$$7 \;/\; 2 \rightarrow 3 \rightarrow 3.00$$

Fix (make one side a floating-point value):
`(float)total / n` or `total / 2.0`

No warning at all. Only testing catches this one.

TIP `7/2` gives `3`, not `3.5`, because it is integer division (relates to Q25 in the mock: use `1.0` instead of `1`).

Bug: scanf without &

```c
int age;
printf("Age: ");
scanf("%d", age);
printf("Next year you are %d\n",
       age + 1);
```

Bug: scanf without &

```c
int age;
printf("Age: ");
scanf("%d", age);
printf("Next year you are %d\n",
       age + 1);
```

```
$ echo 21 | ./rt4_scanf
$ echo $?
139
```

No output at all: the program **crashed** (139 = segmentation fault). Even `Age:` got lost!

Bug: scanf without &

```c
int age;
printf("Age: ");
scanf("%d", age);
printf("Next year you are %d\n",
       age + 1);
```

```
$ echo 21 | ./rt4_scanf
$ echo $?
139
```

No output at all: the program **crashed** (139 = segmentation fault). Even `Age:` got lost!

`scanf` needs the **address** of the variable, so it knows *where* to store the number. Without `&`, it uses the garbage value in `age` as an address and writes to random memory.

Fix: `scanf("%d", &age);`

`-Wall` warned: *format '%d' expects argument of type 'int *', but argument 2 has type 'int'*.

Why was `Age:` lost? Output waits in a **buffer** (especially when it goes to a pipe or file, like an auto-grader), and the crash threw the buffer away. See the print-debugging section.

Bug: off-by-one / array out of bounds

```
1  int a[5] = {10, 20, 30, 40, 50};
2  int sum = 0;
3  for (int i = 0; i <= 5; i++)
4      sum += a[i];
5  printf("sum = %d\n", sum);
```

Expected `150`. **Got:**

TIP An array of size N has valid indices 0 to N-1 only. `a[N]` is outside the array (relates to Q24 in the mock).

Bug: off-by-one / array out of bounds

```c
int a[5] = {10, 20, 30, 40, 50};
int sum = 0;
for (int i = 0; i <= 5; i++)
    sum += a[i];
printf("sum = %d\n", sum);
```

Expected `150`. **Got:**

```
sum = 298
```

TIP An array of size N has valid indices 0 to N-1 only. `a[N]` is outside the array (relates to Q24 in the mock).

Bug: off-by-one / array out of bounds

```c
int a[5] = {10, 20, 30, 40, 50};
int sum = 0;
for (int i = 0; i <= 5; i++)
    sum += a[i];
printf("sum = %d\n", sum);
```

Expected `150`. **Got:**

```
sum = 298
```

`int a[5]` has indices **0 to 4**. The loop also reads `a[5]`, which is *outside* the array: whatever happens to be in memory next.

C does **not** check array bounds. You might get garbage, a crash, or the right answer by luck.

Fix: `for (int i = 0; i < 5; i++)`

Pattern to remember: size N $\Rightarrow$ `i = 0; i < N`.

TIP An array of size N has valid indices 0 to N-1 only. `a[N]` is outside the array (relates to Q24 in the mock).

Bug: a semicolon after `for` / `if`

```
1  int count = 0;
2  for (int i = 0; i < 5; i++);
3  {
4      count++;
5  }
6  printf("count = %d\n", count);
```

Expected 5. **Got:**

Bug: a semicolon after `for` / `if`

```
1  int count = 0;
2  for (int i = 0; i < 5; i++);
3  {
4      count++;
5  }
6  printf("count = %d\n", count);
```

Expected 5. **Got:**

```
count = 1
```

Bug: a semicolon after `for` / `if`

```c
int count = 0;
for (int i = 0; i < 5; i++);
{
    count++;
}
printf("count = %d\n", count);
```

Expected 5. **Got:**

```
count = 1
```

The `;` **is** the loop body: an empty statement, repeated 5 times. The `{ count++; }` block runs **once**, after the loop.

gcc 16 does warn:

```
warning: this 'for' clause does not guard...
  [-Wmisleading-indentation]
```

Fix: delete the `;` after `)`. `if (x > 0);` has exactly the same problem.

Bug: the loop that never ends

```c
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
}
```

TIP If the program hangs with no more output, press Ctrl+C and check whether the variable in the loop condition is ever changed.

Bug: the loop that never ends

```c
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
}
```

```
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
  ...
```

Press **Ctrl+C** to stop a program that is stuck.

TIP If the program hangs with no more output, press Ctrl+C and check whether the variable in the loop condition is ever changed.

Bug: the loop that never ends

```c
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
}
```

```
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
  ...
```

Press **Ctrl+C** to stop a program that is stuck.

Nothing inside the loop changes `i`, so `i <= 5` stays true forever.

Check every loop for 3 things:

1. **start**: is the variable initialized?
2. **condition**: will it become false eventually?
3. **update**: does it move *towards* that?

Fix: add `i++;` inside the loop.

TIP If the program hangs with no more output, press Ctrl+C and check whether the variable in the loop condition is ever changed.

Bug: the dangling `else`

```
1  int marks = 40, attendance = 90;
2  if (marks >= 50)
3      if (attendance >= 80)
4          printf("Pass\n");
5  else
6      printf("Fail\n");
7  printf("done\n");
```

Expected `Fail`. **Got:**

Bug: the dangling `else`

```c
int marks = 40, attendance = 90;
if (marks >= 50)
    if (attendance >= 80)
        printf("Pass\n");
else
    printf("Fail\n");
printf("done\n");
```

Expected `Fail`. **Got:**

```
done
```

Bug: the dangling `else`

```
1  int marks = 40, attendance = 90;
2  if (marks >= 50)
3      if (attendance >= 80)
4          printf("Pass\n");
5  else
6      printf("Fail\n");
7  printf("done\n");
```

Expected `Fail`. **Got:**

```
done
```

An `else` belongs to the **nearest** unmatched `if`, whatever the indentation says. Here it pairs with `if (attendance >= 80)`.

Fix: braces make the meaning explicit:

```
if (marks >= 50) {
    if (attendance >= 80)
        printf("Pass\n");
} else
    printf("Fail\n");
```

`-Wall` warns: *suggest explicit braces to avoid ambiguous 'else'*. (Mock test Q16.)

Print debugging: make the invisible visible

The simplest debugger is `printf`. Add temporary prints to see **what the program really does**, then remove them.

Five tricks in this section

1. wrap values in `[ ]`
2. print `*` instead of a space
3. trace variables in every loop pass
4. `fflush` / `stderr` when it crashes
5. switch debug prints on/off with `-DDEBUG`

Good debug prints

- start with a tag: `[DEBUG]`
- show **name** and **value**: `i=3 sum=6`, not `3 6`
- put them where you *suspect* the bug: in loops, around `if`

TIP Remember to remove debug prints before submitting, because the grader compares output

Trick 1: wrap values in [ ]

```c
printf("Total: %d \n", total);

/* debug: show the edges */
printf("[Total: %d ]\n", total);
printf("[%s]\n", name);
printf("[%5d]\n", total);
```

```
Total: 15
```

Looks perfect, but the grader says wrong answer.

Trick 1: wrap values in [ ]

```c
printf("Total: %d \n", total);

/* debug: show the edges */
printf("[Total: %d ]\n", total);
printf("[%s]\n", name);
printf("[%5d]\n", total);
```

```
Total: 15
```

Looks perfect, but the grader says wrong answer.

```
[Total: 15 ]
[Somchai]
[   15]
```

Now you can see the **trailing space** after `15`, and exactly how wide `%5d` is.

Key idea

Spaces, tabs and newlines are invisible. Brackets show exactly where a value starts and ends. Very handy for format questions like mock Q10.

Trick 2: print * instead of a space

Goal: a right-aligned triangle with `n = 4`. The output looks “a bit off”, but by how much?

```c
for (int i = 1; i <= n; i++)
{
  for (int s = 1; s <= n - i + 1; s++)
    printf(" ");   /* try "*" */
  for (int k = 1; k <= i; k++)
    printf("#");
  printf("\n");
}
```

Wanted

```
   #
  ##
 ###
####
```

Got

```
   #
  ##
 ###
####
```

TIP For pattern problems, temporarily replace spaces with `*` so you can count the spaces in each row at a glance.

Trick 2: print * instead of a space

Goal: a right-aligned triangle with `n = 4`. The output looks “a bit off”, but by how much?

```c
for (int i = 1; i <= n; i++)
{
  for (int s = 1; s <= n - i + 1; s++)
    printf(" ");   /* try "*" */
  for (int k = 1; k <= i; k++)
    printf("#");
  printf("\n");
}
```

Wanted

```
   #
  ##
 ###
####
```

Got

```
   #
  ##
 ###
####
```

With "*"

```
****#
***##
**###
*####
```

TIP For pattern problems, temporarily replace spaces with `*` so you can count the spaces in each row at a glance.

Trick 2: print * instead of a space

Goal: a right-aligned triangle with `n = 4`. The output looks “a bit off”, but by how much?

```c
for (int i = 1; i <= n; i++)
{
  for (int s = 1; s <= n - i + 1; s++)
    printf(" ");   /* try "*" */
  for (int k = 1; k <= i; k++)
    printf("#");
  printf("\n");
}
```

Wanted

```
   #
  ##
 ###
####
```

Got

```
   #
  ##
 ###
####
```

With ”*”

```
****#
***##
**###
*####
```

Count the stars: row 1 has **4** spaces but needs **3**. Row i needs $n - i$ spaces, not $n - i + 1$.
Fix: `s <= n - i`, then put ” ” back.

TIP For pattern problems, temporarily replace spaces with * so you can count the spaces in each row at a glance.

Trick 3: trace the variables in every loop pass

Goal: sum of the digits of 4096 $= 4 + 0 + 9 + 6 = 19$.

```c
int n = 4096, sum = 0;
while (n > 10)
{
    printf("[DEBUG] n=%d  digit=%d  sum=%d\n",
           n, n % 10, sum);
    sum = sum + n % 10;
    n = n / 10;
}
printf("sum = %d\n", sum);
```

```
[DEBUG] n=4096  digit=6  sum=0
[DEBUG] n=409  digit=9  sum=6
[DEBUG] n=40  digit=0  sum=15
sum = 15
```

Trick 3: trace the variables in every loop pass

Goal: sum of the digits of 4096 $= 4 + 0 + 9 + 6 = 19$.

```c
int n = 4096, sum = 0;
while (n > 10)
{
    printf("[DEBUG] n=%d  digit=%d  sum=%d\n",
           n, n % 10, sum);
    sum = sum + n % 10;
    n = n / 10;
}
printf("sum = %d\n", sum);
```

```
[DEBUG] n=4096  digit=6  sum=0
[DEBUG] n=409  digit=9  sum=6
[DEBUG] n=40  digit=0  sum=15
sum = 15
```

The trace stops at `n=40`: the loop never runs for `n=4`, so the digit **4** is never added.

`4 > 10` is false. The condition should be `n > 0`.

Trick 3: trace the variables in every loop pass

Goal: sum of the digits of 4096 $= 4 + 0 + 9 + 6 = 19$.

```c
int n = 4096, sum = 0;
while (n > 10)
{
    printf("[DEBUG] n=%d  digit=%d  sum=%d\n",
           n, n % 10, sum);
    sum = sum + n % 10;
    n = n / 10;
}
printf("sum = %d\n", sum);
```

```
[DEBUG] n=4096  digit=6  sum=0
[DEBUG] n=409  digit=9  sum=6
[DEBUG] n=40  digit=0  sum=15
sum = 15
```

The trace stops at `n=40`: the loop never runs for `n=4`, so the digit **4** is never added.

`4 > 10` is false. The condition should be `n > 0`.

Key idea

Write down what you *expect* each line of the trace to be, then compare. The **first** line that differs points to the bug.

Trick 4: when it crashes, use `fflush` or `stderr`

```c
int age;
printf("Age: ");
fflush(stdout);           /* push it out NOW
   */
fprintf(stderr, "[DEBUG] before scanf\n");
scanf("%d", age);         /* BUG: no & */
fprintf(stderr, "[DEBUG] after scanf\n");
```

```
$ echo 21 | ./pd4_flush
Age: [DEBUG] before scanf
Segmentation fault
```

`printf` output waits in a **buffer** and is written later. If the program crashes first, it's **lost**, so your debug print "never happened".

- `fflush(stdout);` writes the buffer out right now
- `fprintf(stderr, ...)` is **not buffered**, and it doesn't mix into the normal output the grader checks

`before` printed, `after` didn't. So the crash is **between them**: the `scanf` line.

Trick 5: debug prints you switch on and off

```c
for (int i = 1; i <= n; i++)
{
    fact = fact * i;
#ifdef DEBUG
    printf("[DEBUG] i=%d fact=%d\n",
           i, fact);
#endif
}
printf("%d! = %d\n", n, fact);
```

Lines between `#ifdef DEBUG` and `#endif` are only compiled when DEBUG is defined: the preprocessor removes them otherwise.

Normal build: clean output

```
$ gcc pd3_ifdef.c -o fact
$ ./fact
5! = 120
```

Debug build: `-DDEBUG` defines it

```
$ gcc -DDEBUG pd3_ifdef.c -o fact
$ ./fact
[DEBUG] i=1 fact=1
[DEBUG] i=2 fact=2
[DEBUG] i=3 fact=6
[DEBUG] i=4 fact=24
[DEBUG] i=5 fact=120
5! = 120
```

gdb: pause the program and look inside

A **debugger** runs your program **one line at a time** and lets you inspect any variable, with no `printf` needed.

1. Compile with `-g` (keeps line numbers and variable names):

```
$ gcc -g -Wall gdb_avg.c -o avg
$ gdb ./avg
(gdb)
```

Windows
gdb comes with MSYS2

macOS
use `lldb` instead (same idea)

Linux
`gdb` (`apt install gdb`)

Command	What it does
`break 11` (`b`)	stop when line 11 is reached
`run` (`r`)	start the program
`next` (`n`)	run one line (step *over* calls)
`step` (`s`)	run one line (step *into* calls)
`print x` (`p`)	show the value of x
`display x`	show x after every step
`info locals`	show all local variables
`continue` (`c`)	run to the next breakpoint
`quit` (`q`)	leave gdb

A gdb session: why is the average 57.50?

```c
int marks[4] = {60, 75, 80, 75};
int total = 0;
float avg;

for (int i = 1; i < 4; i++)
    total = total + marks[i];

avg = total / 4.0;
printf("average = %.2f\n", avg);
```

```
(gdb) break 11
Breakpoint 1 at 0x1400014e9: file gdb_avg.c, line 11.
(gdb) run
Breakpoint 1, main () at gdb_avg.c:11
11              total = total + marks[i];
(gdb) print marks
$1 = {60, 75, 80, 75}
(gdb) print i
$2 = 1
(gdb) print marks[i]
$3 = 75
```

Expected $(60 + 75 + 80 + 75)/4 = \mathbf{72.50}$.
The program prints **57.50**.

TIP Set a breakpoint in the loop and print `i` on the first iteration, and you will see right away that it starts at 1, not 0.

A gdb session: why is the average 57.50?

```
6   int marks[4] = {60, 75, 80, 75};
7   int total = 0;
8   float avg;
9
10  for (int i = 1; i < 4; i++)
11      total = total + marks[i];
12
13  avg = total / 4.0;
14  printf("average = %.2f\n", avg);
```

Expected $(60 + 75 + 80 + 75)/4 = \mathbf{72.50}$.
The program prints **57.50**.

```
(gdb) break 11
Breakpoint 1 at 0x1400014e9: file gdb_avg.c, line 11.
(gdb) run
Breakpoint 1, main () at gdb_avg.c:11
11              total = total + marks[i];
(gdb) print marks
$1 = {60, 75, 80, 75}
(gdb) print i
$2 = 1
(gdb) print marks[i]
$3 = 75
```

The **first** pass of the loop adds `marks[1]` = 75.
`marks[0]` = 60 is **never added**: $230/4 = 57.5$.
Fix: `for (int i = 0; i < 4; i++)`

TIP Set a breakpoint in the loop and print `i` on the first iteration, and you will see right away that it starts at 1, not 0.

Debugging checklist

It doesn't compile

1. Read the **first** message: file, line, column.
2. Look at that line **and the one before**.
3. Check `;` `{` `}` `(` `)` `"` `"`, spelling, `#include`.
4. Fix one thing, recompile.

It compiles with warnings

Treat each warning as a bug until proven otherwise.

It runs but the answer is wrong

1. Use a **small** input you can check by hand.
2. Usual suspects: `=`/`==`, start values, integer division, `<`/`<=`, format specifiers.
3. Trace (`printf` / `gdb`): find the **first** wrong value.

It crashes / hangs

`&` missing in `scanf`? index out of range? loop variable never updated?